L'EDITOR DI CODICE

# bm Code

Scrivi il Lua del tuo gioco bm direttamente sulla console, eseguilo, fermalo su una riga e leggi i suoi errori.

# Indice

**Cos'è bm Code** . . . . . **4**

Cosa fa . . . . . 4

Perché è importante . . . . . 4

Da sapere . . . . . 4

**Aprire e salvare** . . . . . **5**

Progetti e giochi . . . . . 5

Aprire una cartuccia . . . . . 5

Un nuovo progetto . . . . . 5

Salvare . . . . . 5

Uscire . . . . . 5

**Lo schermo** . . . . . **6**

Font . . . . . 6

Due pagine . . . . . 6

Il menu . . . . . 6

**Tasti di modifica** . . . . . **8**

Muoversi . . . . . 8

Modificare . . . . . 8

Pagine, schede e font . . . . . 8

Eseguire e farsi aiutare . . . . . 9

Con il gamepad . . . . . 9

**Completamento e assistente** . . . . . **10**

Completamento delle parole . . . . . 10

L'assistente (F6) . . . . . 10

Aiuto su una funzione API (Ctrl+Q) . . . . . 10

Il controllo dei nomi API al salvataggio . . . . . 10

**Breakpoint (F8)** . . . . . **12**

Quando il gioco si ferma . . . . . 12

**Suonare riff con Ctrl+Enter** . . . . . **13**

**Eseguire il gioco e leggere gli errori . . . . . 14**

Eseguire . . . . . 14

Quando il gioco fallisce . . . . . 14

**Tutorial guidati . . . . . 15**

Scrivi ed esegui la tua prima riga . . . . . 15

Correggi un errore con la riga dell'errore . . . . . 15

Usa un breakpoint . . . . . 15

**Consigli e limiti . . . . . 17**

**Punti aperti . . . . . 18**

LEGGIMI

**Cos'è questo.** Il manuale di bm Code, l'editor con cui scrivi il codice Lua di un gioco sulla console stessa.
**Come usarlo.** Leggi una volta i primi capitoli, poi segui i tutorial guidati nell'ordine. La tabella dei tasti serve per tornarci sopra.
**Cosa manca.** Ogni lacuna è segnata con un'etichetta DA COMPLETARE evidenziata; nulla è inventato.

# Cos'è bm Code

bm Code è l'editor di codice della suite bm. Apre una cartuccia, mostra il suo codice Lua, ti lascia modificarlo ed esegue il gioco dalla stessa schermata.

## Cosa fa

- Più cartucce aperte insieme, ognuna nella sua scheda, e due pagine si possono mostrare affiancate.
- Tre font: 6x12, 8x14 e 8x16. Il 6x12 contiene 106 colonne.
- Colori della sintassi, annulla e ripeti, cerca e sostituisci, vai alla riga.
- Un completamento delle parole: mentre scrivi compare il resto della parola più probabile e Tab la scrive.
- Un assistente su F6 che risponde alle domande e scrive codice per te.
- Breakpoint su F8 e debugger, con F5 per eseguire.
- Ctrl+Enter suona il codice come musica (riff).
- Ctrl+Q (sul pad Y+X) mostra tutte le firme del nome API sotto il cursore, ciascuna con il suo testo; al salvataggio i nomi che non sono nell'API vengono segnalati e i loro numeri di riga diventano rossi.

## Perché è importante

Puoi scrivere, provare e correggere un gioco senza un computer: il ciclo è modifica, F5, guarda, torna indietro.

## Da sapere

- Cambia solo il **codice** di una cartuccia. Sprite, mappa, suoni e copertina restano come sono.
- bm Code è un programma del Pi Zero W. Il kernel del PowKiddy RGB30 non lo offre; lì scrivi il codice nell'editor rapido del bm SDK (F2).
- Dentro il bm SDK puoi aprire il progetto in bm Code dalla pagina F1 (*OPEN IN*, tasto 1).

# Aprire e salvare

## Progetti e giochi

bm Code modifica solo i **progetti**, i file che finiscono in `.bme`. Un gioco (`.bm`) è in sola lettura: lo puoi aprire e leggere, ma mai cambiare.

Quando salvi un gioco, bm Code non lo tocca. Scrive una copia modificabile in `/bme`, `name.bme`, e da quel momento la scheda lavora sulla copia. Il nome della scheda cambia nel nuovo file.

## Aprire una cartuccia

Premi **Ctrl+O**. L'elenco mostra `+ New project...` e poi i progetti `.bme` di `/bme` e i file `.bm` e `.bme` di `/carts` e della radice della scheda. Muoviti con le frecce su e giù, premi **Enter** per aprire. Una cartuccia già aperta mostra semplicemente la sua scheda.

## Un nuovo progetto

Premi **Ctrl+N**. Una riga chiede `New project, file name (8.3 in /bme):` e suggerisce `game1.bme`. Il nome deve essere 8.3: fino a otto lettere, cifre, `_` o `-`, e l'estensione `.BME`. Se il file esiste già, bm Code lo dice e suggerisce Ctrl+O. La nuova scheda ha un piccolo programma di partenza; premi **Ctrl+S** per scrivere il file.

**Ctrl+T** apre una nuova scheda vuota senza file: il primo salvataggio chiede un nome.

## Salvare

| Tasto | Cosa fa |
|---|---|
| Ctrl+S | salva la scheda; una scheda senza file chiede un nome |
| Ctrl+Shift+S | salva con nome: chiede un nome 8.3 in `/carts` |
| Ctrl+W | chiude la scheda; se ha modifiche chiede `Save`, `Discard` o `Cancel` |

Una scheda con modifiche mostra un `*` dopo il nome. F5 salva ogni scheda con modifiche che ha un file prima di eseguire il gioco; una scheda nuova senza nome non viene salvata.

## Uscire

Scegli **Exit** nel menu (Esc). Con modifiche non salvate chiede `Save all`, `Keep for later` o `Cancel`. **Ctrl+Esc** torna al menu di bm; se ci sono modifiche, premi Ctrl+Esc una seconda volta e vengono conservate per la volta dopo.

Le schede, i cursori, il font e i breakpoint tornano la prossima volta che apri bm Code. Anche il testo non salvato viene conservato, entro un limite: vedi Consigli e limiti.

# Lo schermo

**[DA COMPLETARE: shots/editor-code-screen.png]**

- **Barra in alto**: una scheda per ogni cartuccia, con il nome del file. Un `*` indica modifiche non salvate. A destra, `held: keys` ricorda che F12 tenuto premuto mostra tutti i tasti.
- **Il codice**: i numeri di riga a sinistra, la riga corrente illuminata. Un breakpoint è un numero di riga rosso scuro. La riga di un errore ha lo sfondo rossiccio.
- **Riga di stato** (l'ultima riga): il file, un `*` se modificato, poi `ln` (riga) sul totale, `col` e il font. Durante la musica aggiunge `MUSIC` e il numero di pattern che suonano. I messaggi compaiono accanto al nome del file.

## Font

**F10** cambia il font a rotazione: 6x12, 8x14, 8x16. La riga di stato mostra quello in uso.

## Due pagine

**F4** mostra due pagine affiancate. Ogni pagina ha la sua scheda. **F7** passa da una pagina all'altra; **F2** e **F3** cambiano la scheda della pagina in cui sei. Nella barra in alto una scheda mostra `1` o `2` per la pagina che la mostra. F4 di nuovo torna a una pagina.

## Il menu

**Esc** apre il menu (con una selezione attiva, Esc prima lascia la selezione). Su e giù muovono, Enter sceglie, Esc chiude. Le voci:

| Voce | Tasto |
|---|---|
| New project | Ctrl+N |
| Open... | Ctrl+O |
| Save | Ctrl+S |
| Save as... | Ctrl+Shift+S |
| Close tab | Ctrl+W |
| Run the game | F5 |
| Breakpoint | F8 |
| Debug from the start | |
| Clear breakpoints | |
| Split screen | F4 |
| Font size | F10 |
| Find | Ctrl+F |
| Replace | Ctrl+H |
| Go to line | Ctrl+L |

| Voce | Tasto |
|---|---|
| Assistant | F6 |
| Explain the error | F9 |
| Play the music (riff) | Ctrl+Enter |
| Stop the music | Ctrl+. |
| Word completion | on / off |
| Words in comments | la lingua delle parole |
| Pad typing | lo stato |
| Keys | F12 tenuto |
| Exit | |

Quando è stato l'SDK ad aprire il file, il menu ha anche **Back to bm SDK**: salva e ritorna.

# Tasti di modifica

Questi sono i tasti che esistono in bm Code. **F11** e **F12** appartengono al sistema e non arrivano mai all'editor. Un elenco dei tasti è sullo schermo mentre tieni premuto **F12**.

## Muoversi

| Tasto | Cosa fa |
|---|---|
| Frecce | muovono il cursore |
| Home | va al primo carattere della riga; di nuovo, alla colonna 1 |
| End | va alla fine della riga |
| PgUp / PgDn | si muovono di una pagina |
| Ctrl+L | vai alla riga: scrivi il numero |
| Ctrl+F | cerca: scrivi il testo (la parola sotto il cursore è suggerita) |
| Ctrl+G | cerca il successivo |
| Ctrl+H | sostituisci tutto: chiede il testo, poi il nuovo testo |

## Modificare

| Tasto | Cosa fa |
|---|---|
| Enter | nuova riga, con il rientro di quella sopra (due in più dopo `then`, `do`, `else`, `function ...` e simili) |
| Backspace / Del | cancellano prima / dopo il cursore |
| Tab | scrive il completamento grigio-azzurro se c'è; altrimenti rientra la riga (o le righe selezionate) di due spazi |
| Ctrl+U | toglie il rientro della riga (o delle righe selezionate) |
| Ctrl+B | inizia una selezione; muovi il cursore per estenderla; Ctrl+B di nuovo la lascia |
| Ctrl+C / Ctrl+X | copiano / tagliano la selezione |
| Ctrl+V | incolla; una riga intera va sopra quella corrente |
| Ctrl+K | taglia l'intera riga |
| Ctrl+D | duplica la riga |
| Ctrl+Z / Ctrl+Y | annulla / ripeti |

Scrivere `end`, `else`, `elseif`, `until` o `}` da soli su una riga li riporta al rientro del loro blocco.

## Pagine, schede e font

| Tasto | Cosa fa |
|---|---|
| Ctrl+T | nuova scheda vuota |
| Ctrl+W | chiude la scheda |
| F2 / F3 | scheda precedente / successiva |
| F4 | una pagina / due pagine |
| F7 | passa all'altra pagina |
| F10 | font 6x12 / 8x14 / 8x16 |

## Eseguire e farsi aiutare

| Tasto | Cosa fa |
|---|---|
| F5 o Ctrl+R | esegue il gioco |
| F8 | breakpoint sulla riga |
| F6 | l'assistente |
| F9 | spiega l'errore del gioco |
| Ctrl+Q | aiuto sulla funzione API sotto il cursore |
| Ctrl+Enter | suona il codice come musica (riff) |
| Ctrl+. | ferma la musica |
| Esc | lascia la selezione, oppure apre il menu |
| Ctrl+Esc | torna al menu di bm |
| F12 tenuto | i tasti di bm Code, sotto quelli del sistema |

## Con il gamepad

| Pulsante | Cosa fa |
|---|---|
| D-pad | muove il cursore |
| Y + sinistra / destra | scheda precedente / successiva |
| Y + su / giù | pagina su / pagina giù |
| X | l'assistente |
| Y + X | aiuto sulla funzione API sotto il cursore |
| Start | il menu |

Nel menu e negli elenchi il D-pad muove, A sceglie e B torna indietro. Il pad può anche scrivere testo: attiva **Pad typing** nel menu. Select la attiva e la disattiva; la voce di menu chiama Share lo stesso pulsante.

# Completamento e assistente

## Completamento delle parole

Mentre scrivi una parola, il resto della parola più probabile compare in grigio-azzurro dopo il cursore. La riga di stato mostra `Tab:` e la parola, e le altre scelte dopo `also:`. Premi **Tab** e la parola viene scritta (verde fino al tasto successivo). Continua a scrivere e sparisce.

Le parole dipendono da dove sei:

- nel codice: le parole di Lua, le funzioni di bm e i nomi che sono nella scheda;
- in un commento (dopo `--` ) e in una stringa: parole comuni, in italiano o in inglese;
- dopo `#entry:` (vedi sotto): italiano e le domande per l'assistente.

Nel menu, **Word completion** lo attiva e lo disattiva, e **Words in comments** cambia la lingua delle parole comuni. Entrambi sono conservati per la volta dopo.

## L'assistente (F6)

**F6** apre il pannello dell'assistente in modalità codice, con la parola sotto il cursore come contesto. Risponde mentre scrivi la domanda. **Enter** inserisce il codice che propone nel punto del cursore, **Esc** lo chiude. Niente gira mentre è chiuso. L'inserimento è un solo passo di annullamento con Ctrl+Z.

In modalità codice l'assistente risponde con quattro tipi di voci della sua base di conoscenza: funzioni **API**, esempi **how-to** con codice, messaggi di **errore** di Lua e **consigli**. Le sue risposte su sprite, modelli 3D e musica sono negli altri strumenti.

### Una riga `#entry:`

Puoi chiedere qualcosa senza aprire il pannello. Scrivi una riga come questa e premi **Enter** su di essa:

```
#entry: ... quello che vuoi ... #
```

L'assistente lo fa sul posto e la riga di stato dice cosa ha fatto, con `(Ctrl+Z undoes)` . Se non sa nulla per quella richiesta, la riga di stato lo dice e il testo resta.

### Spiegare un errore

**F9** apre l'assistente sull'errore dell'ultima esecuzione. Se non c'è errore, la riga di stato dice `no error: F5 runs the game, F9 explains its error` .

## Aiuto su una funzione API (Ctrl+Q)

Porta il cursore sul nome di una funzione di bm e premi **Ctrl+Q** (sul pad, tieni premuto **Y** e premi **X**). Si apre una finestra con ogni firma del nome, ciascuna seguita dal suo testo (un nome può avere più firme). In fondo la finestra elenca i file del manuale dell'API che descrivono il nome ( `docs/api/NAME.md` ), una riga per file. Qualsiasi tasto chiude la finestra.

- Su un nome che non è nell'API, la riga di stato dice `no API help for 'NAME'` .
- Senza nessun nome sotto il cursore, dice `no name under the cursor` .
- Con il mouse, il tasto destro apre un menu che contiene *API help*.

## Il controllo dei nomi API al salvataggio

Quando salvi una scheda il cui file finisce in `.bm` , `.bme` o `.lua` , bm Code cerca i nomi che sembrano chiamate all'API di bm ma non ci sono, come fa `scripts/api_lint.py` .

- La riga di stato dice `API: N unknown name(s), line L: NAME` per il primo, e il numero di ogni riga con un nome sconosciuto diventa **rosso**.
- Un nome che definisci tu, in questa scheda o in un'altra scheda aperta, vale come noto.
- I numeri rossi spariscono appena cambi il testo (anche con annulla o ripeti). Salva di nuovo per controllare di nuovo.
- Una riga con breakpoint mantiene il suo numero rosso scuro.

**PROVA**

Premi **Ctrl+T** per una scheda vuota e scrivi `cls(1)`. Porta il cursore su `cls` e premi **Ctrl+Q**: vedi `cls([c])` e cosa fa. Premi un tasto qualsiasi. Ora scrivi `sprt(1)` su una riga e premi **Ctrl+Q**: la riga di stato dice `no API help for 'sprt'`.

# Breakpoint (F8)

Un breakpoint fa fermare il gioco su una riga, così puoi guardarci dentro.

- **F8** su una riga ci mette un breakpoint: il suo numero di riga diventa rosso scuro e la riga di stato dice `breakpoint on line N`. F8 di nuovo lo toglie.
- **Clear breakpoints** nel menu li rimuove tutti dalla scheda.
- **Debug from the start** nel menu esegue il gioco e lo ferma alla prima riga.
- I breakpoint seguono le loro righe quando inserisci o cancelli righe sopra di loro. Se cancelli la riga, il suo breakpoint sparisce.
- I breakpoint sono conservati con la sessione.
- Nel codice di un gioco, `breakpoint()` ferma il debugger su quella riga quando il gioco è eseguito da bm Code. Negli altri giochi non fa nulla.

## Quando il gioco si ferma

Il debugger copre lo schermo. Mostra il codice attorno alla riga, le variabili della funzione (le locali, poi gli upvalue in azzurro) e lo stack.

| Tasto | Cosa fa |
|---|---|
| F10 o A | riga successiva (senza entrare nelle chiamate) |
| F8 o X | dentro la chiamata |
| Shift+F8 o Y | fuori dalla funzione |
| F5 o Start | continua |
| Esc o Select | ferma il gioco |
| Frecce | scorrono le variabili e scelgono la funzione dello stack |

Quando fermi il gioco, bm Code torna con il cursore sulla riga e il messaggio `stopped in the debugger at line N`.

# Suonare riff con Ctrl+Enter

riff è la notazione musicale di bm (vedi il manuale riff). In bm Code la scrivi e la senti subito.

- In una scheda che non contiene nessun blocco `code [[ ]]`, **Ctrl+Enter** suona l'intera scheda.
- In un gioco, suona il blocco `riff.code [[ ... ]]` sotto il cursore, oppure il primo se il cursore non è in un blocco.
- Le parole delle note che suonano si **illuminano**, finché il testo è quello che è stato suonato.
- La riga di stato dice `MUSIC` e il numero di pattern che suonano.
- **Ctrl+.** ferma la musica. Il menu ha le stesse due voci: *Play the music (riff)* e *Stop the music*.
- Se il riff ha un errore, la sua riga è segnata e la riga di stato dice perché. La musica che suonava continua.
- Se non suona nulla, la riga di stato suggerisce di dare un pattern a un nome, per esempio `drums = s "kick*4"`.
- Eseguire il gioco con F5 ferma la musica.

# Eseguire il gioco e leggere gli errori

## Eseguire

**F5** (o **Ctrl+R**, o *Run the game* nel menu) salva le schede con modifiche e suona il gioco della scheda corrente. La scheda deve avere un file: una scheda nuova senza nome ti chiede prima di salvare ( `save it first (Esc, Save as)` ).

Per uscire dal gioco premi **Start + Select** (o **Ctrl+Esc**). bm Code torna dove eri, con `back from the game` nella riga di stato.

## Quando il gioco fallisce

Se il gioco si ferma con un errore, bm Code torna e:

- mette il cursore sulla riga dell'errore (letta da `main.lua:N:` nel messaggio);
- colora quella riga con lo sfondo rossiccio;
- mostra `the game stopped:` e il messaggio nella riga di stato, seguiti da `(F9: explain)` .

Premi **F9** per far spiegare l'errore all'assistente. Il segno sparisce alla prossima esecuzione del gioco.

> **NOTA**
> Il numero di riga viene dal messaggio. Se il messaggio non ha una riga, il cursore resta dove era.

# Tutorial guidati

Parti da bm Code. Sulla console è la voce **bm Code** del menu di bm. Puoi anche partire dal bm SDK: F1, poi 1 (*OPEN IN* bm Code).

## Scrivi ed esegui la tua prima riga

1. Avvia bm Code. Al primo avvio mostra un piccolo programma di partenza in una scheda vuota.
2. Premi **Ctrl+N**. Una riga chiede il nome del file e suggerisce `game1.bme`. Premi **Enter**. La riga di stato dice `new: Ctrl+S writes /bme/game1.bme`.
3. Premi **Ctrl+S**. La riga di stato dice `saved /bme/game1.bme`.
4. Porta il cursore con le frecce sulla riga `print("hello!", 296, 40, 0xFFFFFF)`. Cambia `hello!` con le tue parole. Il `*` compare dopo il nome del file.
5. Premi **F5**. Il gioco parte: un cerchio giallo, le tue parole sullo schermo, e il D-pad muove il cerchio.
6. Premi **Start + Select** (o **Ctrl+Esc**) per uscire. bm Code torna con `back from the game`.

**[DA COMPLETARE: shots/editor-code-first-run.png]**

> **PROVA**
> Aggiungi una riga sotto il `print`: `print("score 0", 296, 60, 0xFFFFFF)`. Premi F5 e cercala sullo schermo.

## Correggi un errore con la riga dell'errore

1. Con il progetto del primo tutorial aperto, trova la riga `circfill(x, y, 12, 0xFFD050)` in `_draw`.
2. Rompila apposta: togli l'ultima lettera di `circfill`, così diventa `circfil(x, y, 12, 0xFFD050)`.
3. Premi **F5**. Il gioco si ferma appena disegna.
4. bm Code torna. Il cursore è sulla riga rotta, che ha lo sfondo rossiccio, e la riga di stato dice `the game stopped:` e il messaggio, con `(F9: explain)`.
5. Premi **F9**. L'assistente spiega l'errore. **Esc** lo chiude.
6. Rimetti la lettera: `circfill`. Premi **F5**. Il cerchio è tornato e il segno è sparito.

**[DA COMPLETARE: shots/editor-code-error-line.png]**

> **ATTENZIONE**
> Se non hai cambiato il programma di partenza, la riga rotta è la riga 16.

## Usa un breakpoint

1. Con il progetto del primo tutorial aperto, porta il cursore sulla riga `if btn(0) then x = x - 2 end` in `_update`.
2. Premi **F8**. Il numero di riga diventa rosso scuro e la riga di stato dice `breakpoint on line N: F5 runs the game, it stops there`.

3. Premi **F5**. Il gioco parte e si ferma su quella riga. Il debugger copre lo schermo: il codice attorno alla riga, le variabili, lo stack.
4. Premi **F10** per andare alla riga successiva. Guarda le variabili cambiare.
5. Premi **F5** per continuare. `_update` gira a ogni fotogramma, quindi si ferma di nuovo sulla stessa riga.
6. Premi **Esc** per fermare il gioco. bm Code torna con il cursore su quella riga e dice `stopped in the debugger at line N`.
7. Premi **F8** sulla riga per togliere il breakpoint.

**[DA COMPLETARE: shots/editor-code-debugger.png]**

PROVA

Metti un breakpoint dentro `_draw`, premi F5 e premi F10 riga per riga. Quale variabile cambia?

# Consigli e limiti

- **Salva con un nome corto.** Il nome del file di un progetto è 8.3: `mygame.bme`.
- **Lavora sulla copia.** Se apri un gioco (`.bm`), il primo salvataggio crea `name.bme`; continua a modificare quello.
- **Token.** I token di un programma sono contati come li conta il dev kit; `code_tokens(text)` dà il numero per un pezzo di Lua. Sono un'informazione, non un limite: bm non mette nessun tetto ai token. bm Code non li mostra: li mostra il bm SDK, nella pagina F1 e nella riga di stato del suo editor rapido.
- **Il testo non salvato è conservato in uno spazio limitato.** Quando esci o esegui il gioco, bm Code conserva per la volta dopo il testo delle schede con modifiche (e delle schede senza file), fino a 16000 caratteri in tutto; il testo oltre non viene conservato. Salva spesso.
- **Annulla** ricorda gli ultimi 100 passi di ogni scheda.
- **Schede.** La barra in alto mostra tante schede quante ne stanno; le altre sono indicate da `...`. Usa F2 e F3 per raggiungerle.
- **Debug solo da bm Code.** `breakpoint()` ferma solo in un gioco eseguito da uno strumento come bm Code.
- **Righe sullo schermo.** Con una pagina sullo schermo 640x360, il codice mostra 28 righe con il font 6x12, 23 con l'8x14 e 20 con l'8x16 (le righe dello schermo, meno la barra in alto e la riga di stato). Le colonne sono 106, 80 e 80. Questi numeri vengono dall'impaginazione nel codice, non da un conteggio sullo schermo. La casella di scrittura con il pad occupa alcune righe.
- **[DA COMPLETARE: il file più grande che bm Code può aprire; il codice non pone un limite proprio e nessuno è stato misurato]**

DA COPIARE

```
F5          -- esegue il gioco
F8          -- breakpoint sulla riga
F9          -- spiega l'errore
Ctrl+Enter  -- suona il riff
```

# Punti aperti

- **Lo schermo**: immagine da fare: shots/editor-code-screen.png
- **Tutorial guidati**: immagine da fare: shots/editor-code-first-run.png
- **Tutorial guidati**: immagine da fare: shots/editor-code-error-line.png
- **Tutorial guidati**: immagine da fare: shots/editor-code-debugger.png
- **Consigli e limiti**: il file più grande che bm Code può aprire; il codice non pone un limite proprio e nessuno è stato misurato